

รายงานงานปฏิบัติการกลุ่ม

การออกแบบและวิเคราะห์อัลกอริทึมด้วย Stack โดยใช้ภาษา Java

กลุ่มที่ 2: Browser Navigation — Two Stacks และ ArrayList

รายชื่อสมาชิกกลุ่ม

นายธนโชติ กสิกิจกรกุล 68122250097
นายณัฐดนัย ลิ้มธรรมรงค์ 66122250037
นายพงษ์นรินทร์ โคตรวงษ์ทอง 68122250015
นายภูวสิษฐ์ ปั่นดี 67122250042
นายমনเทียร พุทธเสน 68122250063
นายณัฐวุฒิ ชมภูวิเศษ 67122250014
นางสาวพัชรภรณ์ ไปสูงเนิน 67122250065

[ชื่อรายวิชา/ อาจารย์ผู้สอน]

วันที่ 16 สิงหาคม 2569

1. บทนำและวัตถุประสงค์

รายงานฉบับนี้จัดทำขึ้นเพื่อวิเคราะห์และออกแบบอัลกอริทึมสำหรับระบบ Browser Navigation ซึ่งต้องรองรับการเปิดหน้าเว็บใหม่ การย้อนกลับ (Back) และการไปข้างหน้า (Forward) โดยเปรียบเทียบวิธีการ 2 แบบ ได้แก่ Two-Stack Method และ ArrayList พร้อม Current Index Method ครอบคลุมตั้งแต่การวิเคราะห์ปัญหา การออกแบบ Pseudocode การพิสูจน์ความถูกต้อง การวิเคราะห์ความซับซ้อนเชิงเวลาและพื้นที่ การพัฒนาโปรแกรมภาษา Java และการทดลองเปรียบเทียบประสิทธิภาพจริง

Case Story

ระบบ Browser ต้องรองรับการเปิดหน้าเว็บไซต์ใหม่ การย้อนกลับไปยังหน้าก่อนหน้า และการไปข้างหน้า เมื่อผู้ใช้กด Back แล้วเปิดหน้าใหม่ ประวัติของหน้า Forward ต้องถูกล้างทิ้งทันที เนื่องจากสายการเข้าชม (branch) เดิมไม่ถูกต้องอีกต่อไป

2. การวิเคราะห์ปัญหา

2.1 Input ของปัญหา

- คำสั่งต่อเนื่องทีละคำสั่ง: VISIT <pageId> [title] [url], BACK, FORWARD, CURRENT, DISPLAY HISTORY
- ข้อมูลของแต่ละหน้าเว็บ (Page): Page ID, Page Title, URL, Visited Time

2.2 Output ที่ต้องการ

- สถานะหลังทุกคำสั่ง: Current Page, Back History (ลำดับจากใกล้ไปไกล), Forward History
- ค่าความสำเร็จ/ล้มเหลวของคำสั่ง BACK และ FORWARD (true/false)

2.3 ข้อจำกัดของปัญหา (Constraints)

- ไม่สามารถ BACK ได้เมื่อไม่มีหน้าก่อนหน้า และไม่สามารถ FORWARD ได้เมื่อไม่มีหน้าถัดไป
- การ VISIT หน้าใหม่ขณะที่มี Forward History อยู่ ต้องล้าง Forward History ทิ้งทั้งหมดทันที
- จำนวนหน้าในประวัติอาจมีได้ตั้งแต่ 0 ถึงหลักแสนหน้า (ทดสอบถึง 100,000 หน้า)

2.4 Assumptions

- การเปิดหน้าเดิมซ้ำถือเป็นการ VISIT ใหม่ตามพฤติกรรมจริงของ Browser (ไม่ใช่การข้ามไปหาหน้าเดิมใน history)
- pageId ห้ามว่าง และถือว่าไม่จำเป็นต้องไม่ซ้ำกันตลอดประวัติ (สามารถเปิดหน้าเดิมได้หลายครั้ง)

2.5 กรณีปกติ (Normal Cases)

- VISIT ต่อเนื่องหลายหน้า แล้ว BACK/FORWARD สลับกันตามปกติ

2.6 กรณีขอบเขต (Boundary Cases)

- BACK เมื่อ Back History ว่าง, FORWARD เมื่อ Forward History ว่าง
- ประวัติมีเพียง 1 หน้า หรือไม่มีหน้าใดเลย (ยังไม่เคย VISIT)
- ประวัติมีขนาดใหญ่มาก ($n = 100,000$ หน้า)

2.7 กรณีที่อาจทำให้โปรแกรมผิดพลาด

- การส่ง pageId ว่างหรือ null เข้ามาใน VISIT
- การเรียก CURRENT ก่อนที่จะเคย VISIT หน้าใดเลย
- คำสั่งที่พิมพ์ผิดรูปแบบหรือไม่รู้จัก

2.8 Operation ที่เกิดขึ้นบ่อยที่สุด

VISIT และ BACK เป็น operation ที่เกิดขึ้นบ่อยที่สุดตามพฤติกรรมการใช้งาน Browser ทั่วไป

2.9 Operation ที่อาจใช้เวลามากที่สุด

การ VISIT หน้าใหม่ทันทีหลังจากมี Forward History ขนาดใหญ่ (จากการ BACK มาหลายครั้ง) เพราะต้องล้าง/ตัด Forward History ทั้งหมดทิ้ง ซึ่งในกรณีเลวร้ายที่สุดมีขนาดถึง $O(n)$

3. การออกแบบอัลกอริทึม

3.1 Algorithm A: Two-Stack Method

1) ชื่ออัลกอริทึม

Two-Stack Browser History

2) แนวคิดหลัก

ใช้ backStack เก็บหน้าที่มาก่อน currentPage และ forwardStack เก็บหน้าที่ถูก Back ออกมา เมื่อ VISIT หน้าใหม่ให้ push currentPage เติมลง backStack แล้วล้าง forwardStack ทั้งทั้งหมด เมื่อ BACK ให้ push currentPage ลง forwardStack แล้ว pop จาก backStack มาเป็น currentPage ใหม่ (สลับบทบาทกันเมื่อ FORWARD)

3) Input

ลำดับคำสั่ง VISIT/BACK/FORWARD พร้อมข้อมูล Page

4) Output

currentPage, backStack, forwardStack หลังแต่ละคำสั่ง

5) Pseudocode

```
ALGORITHM Visit(backStack, forwardStack, currentPage, newPage)
INPUT: currentPage (อาจเป็น null), newPage
OUTPUT: currentPage ใหม่ = newPage, forwardStack ถูกล้าง
1. IF currentPage is not null
```

```

2.     backStack.push(currentPage)
3. END IF
4. WHILE forwardStack is not empty
5.     forwardStack.pop()           // ล้างทิ้งทั้งหมด
6. END WHILE
7. currentPage <- newPage
8. RETURN currentPage

```

ALGORITHM Back(backStack, forwardStack, currentPage)

OUTPUT: true หากย้อนกลับได้สำเร็จ, false หากไม่มีหน้าก่อนหน้า

```

1. IF backStack is empty
2.     RETURN false
3. END IF
4. IF currentPage is not null
5.     forwardStack.push(currentPage)
6. END IF
7. currentPage <- backStack.pop()
8. RETURN true

```

ALGORITHM Forward(backStack, forwardStack, currentPage)

OUTPUT: true หากไปข้างหน้าได้สำเร็จ, false หากไม่มีหน้าถัดไป

```

1. IF forwardStack is empty
2.     RETURN false
3. END IF
4. IF currentPage is not null
5.     backStack.push(currentPage)
6. END IF
7. currentPage <- forwardStack.pop()
8. RETURN true

```

6) ตัวอย่างการทำงาน

ดูหัวข้อ 4. ตัวอย่างการทำงานแบบ Step-by-step

7) ข้อดี

- BACK และ FORWARD ทำงาน $O(1)$ เสมอ ไม่ขึ้นกับขนาดประวัติ
- โครงสร้างเข้าใจง่าย ตรงกับพฤติกรรมจริงของ Browser ที่เป็นการ push/pop สองฝั่ง

8) ข้อจำกัด

- การ VISIT หลัง Back มาไกล ต้องล้าง forwardStack ซึ่งในกรณีเลวร้ายที่สุดใช้เวลา $O(n)$
- ไม่สามารถเข้าถึงหน้ากลาง ๆ ของประวัติแบบสุ่ม (random access) ได้โดยตรง

9) Time Complexity

VISIT: $O(1)$ amortized / $O(n)$ worst-case, BACK: $O(1)$, FORWARD: $O(1)$

10) Space Complexity

$O(n)$ สำหรับเก็บหน้าทั้งหมดใน backStack + forwardStack รวมกัน

3.2 Algorithm B: ArrayList and Current Index Method

1) ชื่ออัลกอริทึม

ArrayList with Current-Index Browser History

2) แนวคิดหลัก

เก็บหน้าทั้งหมดที่เคยเข้าชมในสายปัจจุบันไว้ใน historyList เรียงตามเวลา และใช้ currentIndex ชี้ตำแหน่งหน้าปัจจุบัน หน้าที่มี index มากกว่า currentIndex คือ Forward History เมื่อ VISIT หน้าใหม่ ต้องตัด (truncate) ทุก element ที่มี index มากกว่า currentIndex ที่ก่อน แล้วจึงเพิ่มหน้าใหม่ต่อท้ายและเลื่อน currentIndex

3) Input

ลำดับคำสั่ง VISIT/BACK/FORWARD พร้อมข้อมูล Page

4) Output

historyList, currentIndex หลังแต่ละคำสั่ง

5) Pseudocode

```
ALGORITHM Visit(historyList, currentIndex, newPage)
OUTPUT: currentIndex ใหม่ชี้ไปที่ newPage, ส่วนท้ายที่เกิน currentIndex เดิมถูกตัดทิ้ง
1. removedCount <- size(historyList) - (currentIndex + 1)
2. IF removedCount > 0
3.     ลบ element ตั้งแต่ index (currentIndex+1) ถึงท้าย list
4. END IF
5. historyList.add(newPage)
6. currentIndex <- currentIndex + 1
7. RETURN currentIndex

ALGORITHM Back(historyList, currentIndex)
OUTPUT: true หากย้อนกลับได้สำเร็จ, false หากไม่มีหน้าก่อนหน้า
1. IF currentIndex <= 0
2.     RETURN false
3. END IF
4. currentIndex <- currentIndex - 1
5. RETURN true

ALGORITHM Forward(historyList, currentIndex)
OUTPUT: true หากไปข้างหน้าได้สำเร็จ, false หากไม่มีหน้าถัดไป
1. IF currentIndex >= size(historyList) - 1
2.     RETURN false
3. END IF
4. currentIndex <- currentIndex + 1
5. RETURN true
```

6) ตัวอย่างการทำงาน

ดูหัวข้อ 4. ตัวอย่างการทำงานแบบ Step-by-step

7) ข้อดี

- โครงสร้างข้อมูลเดี่ยว เข้าใจและ debug ง่าย สามารถเข้าถึงหน้าใด ๆ ในประวัติด้วย index ได้โดยตรง (random access $O(1)$)

8) ข้อจำกัด

- การ VISIT หลัง Back มาไกล ต้องลบช่วง (removeRange) ซึ่งเป็น $O(k)$ เช่นกัน และในการทดลองจริงพบว่ามี overhead ด้านการจัดการหน่วยความจำภายในสูงกว่า Two-Stack อย่างชัดเจน

9) Time Complexity

VISIT: $O(1)$ amortized / $O(n)$ worst-case, BACK: $O(1)$, FORWARD: $O(1)$

10) Space Complexity

$O(n)$ สำหรับเก็บ historyList ทั้งหมด (เฉพาะสายปัจจุบัน ไม่เก็บซ้ำสองชุดเหมือน Two-Stack)

หมายเหตุ: ทั้งสองอัลกอริทึมแก้ปัญหเดียวกันและให้ผลลัพธ์ถูกต้องตรงกันทุกกรณี ได้รับการยืนยันด้วย Test Cases ในหัวข้อ 8 ซึ่งรันทั้งสองอัลกอริทึมด้วย input ชุดเดียวกันแล้วเปรียบเทียบผลลัพธ์

4. ตัวอย่างการทำงานแบบ Step-by-step

4.1 กรณีปกติ (Algorithm A: Two-Stack)

ชุดคำสั่งบังคับ: VISIT A, VISIT B, VISIT C, BACK, BACK, FORWARD, VISIT D, BACK, FORWARD

Step	คำสั่ง	Current	Back Stack (ใกล้→ไกล)	Forward Stack (ใกล้→ไกล)
1	VISIT A	A	(empty)	(empty)
2	VISIT B	B	[A]	(empty)
3	VISIT C	C	[B, A]	(empty)
4	BACK	B	[A]	[C]
5	BACK	A	(empty)	[B, C]
6	FORWARD	B	[A]	[C]
7	VISIT D	D	[B, A]	(empty) ← ล้าง Forward ทิ้ง
8	BACK	B	[A]	[D]
9	FORWARD	D	[B, A]	(empty)

ผลลัพธ์นี้ได้จากการรันโปรแกรมจริงผ่านคำสั่ง DEMO ในเมนู Main และ Algorithm B ให้ผลลัพธ์ Current/History เดียวกันทุก step (สลับดูได้ด้วยคำสั่ง SWITCH A|B)

4.2 กรณีขอบเขต: BACK เมื่อไม่มีหน้าก่อนหน้า

Step	คำสั่ง	ผลลัพธ์	Current	หมายเหตุ
1	VISIT A	-	A	เปิดหน้าแรก
2	BACK	false	A	Back Stack ว่าง → ทำไม่ได้

Step	คำสั่ง	ผลลัพธ์	Current	หมายเหตุ
				ค่า Current ไม่เปลี่ยน

5. การอธิบายความถูกต้อง (Correctness Argument)

เลือกอธิบายความถูกต้องของ Algorithm A: Two-Stack Method ด้วยแนวทาง Precondition/Postcondition ร่วมกับ Loop Invariant (สำหรับขั้นตอนล่าง forwardStack ใน VISIT)

5.1 สภาพข้อมูลก่อนเริ่มอัลกอริทึม (Precondition)

backStack และ forwardStack เป็น Stack ที่ถูกต้องตามโครงสร้าง (LIFO) และ currentPage คือหน้าปัจจุบัน หรือ null หากยังไม่เคย VISIT มาก่อน โดยลำดับใน backStack สะท้อนลำดับเวลาการเข้าชมจริงของหน้าในสายปัจจุบัน

5.2 คุณสมบัติที่ยังคงเป็นจริงระหว่างทำงาน (Invariant)

สำหรับ VISIT: ก่อนเข้ารอบ WHILE แต่ละรอบของการล้าง forwardStack เพื่อ pop ทั้ง element ที่ยังไม่ถูกลบทั้งหมดยังคงอยู่ใน forwardStack ครบถ้วนไม่มีการสูญหายหรือสลับลำดับ และ backStack/currentPage ไม่ถูกแก้ไขระหว่างการวนลูปนี้

- Initialization: ก่อนวนลูปครั้งแรก forwardStack มี element ครบตามที่สะสมไว้จากการ BACK ก่อนหน้า
- Maintenance: ทุกครั้งที่ pop 1 ตัวออกจาก forwardStack จำนวน element ที่เหลือลดลง 1 แต่ element ที่เหลือยังคงลำดับเดิม (Stack property คงอยู่)
- Termination: ลูปหยุดเมื่อ forwardStack ว่าง (isEmpty() = true) ซึ่งเกิดขึ้นแน่นอนเพราะขนาดจำกัดและลดลงทุกรอบ

5.3 ผลลัพธ์เมื่ออัลกอริทึมหยุด (Postcondition)

เมื่อ VISIT จบการทำงาน: currentPage เท่ากับ newPage, forwardStack ว่างเปล่า, และ backStack มี currentPage เดิม (ถ้ามี) เพิ่มเข้ามาที่ตำแหน่งบนสุด โดยลำดับ element เดิมใน backStack ไม่เปลี่ยนแปลง ซึ่งตรงตามข้อกำหนดของปัญหาที่ต้องล้าง Forward History เมื่อมีการเปิดหน้าใหม่

5.4 ข้อมูลที่ไม่เกี่ยวข้องไม่สูญหายหรือเปลี่ยนลำดับ

การ push/pop ทุกครั้งกระทำที่ปลายด้านบนของ Stack เพียงด้านเดียว (LIFO) จึงไม่มีการแทรกหรือสลับตำแหน่งของ element อื่นที่ไม่เกี่ยวข้องกับการ operation นั้น ๆ ลำดับของ backStack ที่เหลือหลัง BACK/FORWARD ยังคงสะท้อนลำดับเวลาที่ถูกต้องเสมอ

5.5 การยุติของอัลกอริทึม

ทุก operation (VISIT, BACK, FORWARD) ประกอบด้วย การ push/pop จำนวนจำกัด (ไม่เกิน n ครั้ง โดย n คือขนาดของ forwardStack หรือ backStack ในขณะนั้น) จึงยุติเสมอในเวลาจำกัด ไม่มีการวนลูปไม่รู้จบ

6. การวิเคราะห์ประสิทธิภาพ (Time & Space Complexity)

การวิเคราะห์ครอบคลุมทุกฟังก์ชันของระบบ ไม่ใช่เฉพาะ push()/pop() ที่เป็น $O(1)$ เท่านั้น เนื่องจาก VISIT มีขั้นตอนล่าง/ตัด Forward History ซึ่งขึ้นกับขนาดของ Forward History ในขณะนั้น

Function	Algorithm	Best Case	Average Case	Worst Case	Auxiliary Space
VISIT	A: Two-Stack	$O(1)$	$O(1)$ amortized	$O(n)$	$O(1)$ ต่อครั้ง
VISIT	B: ArrayList	$O(1)$	$O(1)$ amortized	$O(n)$	$O(1)$ ต่อครั้ง
BACK	A: Two-Stack	$O(1)$	$O(1)$	$O(1)$	$O(1)$
BACK	B: ArrayList	$O(1)$	$O(1)$	$O(1)$	$O(1)$
FORWARD	A: Two-Stack	$O(1)$	$O(1)$	$O(1)$	$O(1)$
FORWARD	B: ArrayList	$O(1)$	$O(1)$	$O(1)$	$O(1)$
CURRENT	ทั้งสองวิธี	$O(1)$	$O(1)$	$O(1)$	$O(1)$
DISPLAY HISTORY	ทั้งสองวิธี	$O(1)$	$O(n)$	$O(n)$	$O(n)$ (ใช้พิมพ์)

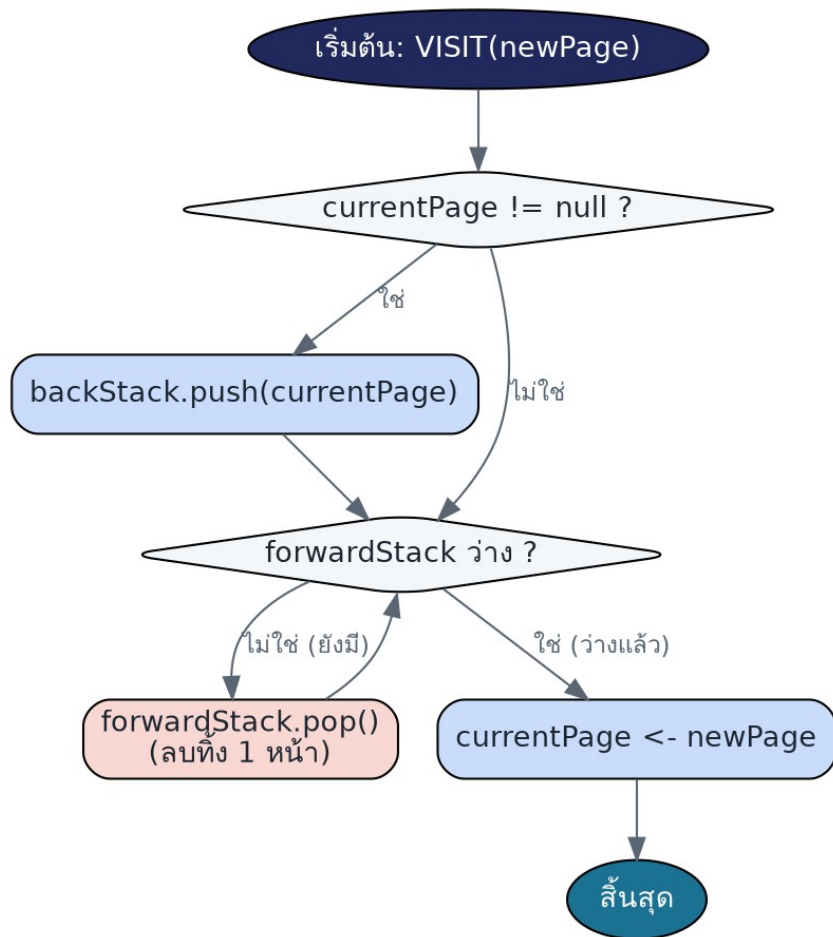
Total Space Complexity: $O(n)$ สำหรับทั้งสองอัลกอริทึม โดย Two-Stack ใช้พื้นที่รวมของ backStack+forwardStack = n เสมอ ส่วน ArrayList เก็บเฉพาะ element ในสายปัจจุบัน (ไม่รวม element ที่ถูกตัดทิ้งไปแล้ว) ทำให้ในทางปฏิบัติมักใช้พื้นที่หน่วยความจำต่อขณะน้อยกว่าเล็กน้อย แต่ทั้งคู่ยังคงอยู่ใน order $O(n)$

เหตุผลที่ VISIT เป็น $O(1)$ amortized ทั้งที่ worst-case คือ $O(n)$: แต่ละ page ที่ถูก push เข้า forwardStack (หรือถูกเพิ่มต่อท้าย historyList) จะถูก pop/ลบทิ้งได้ อย่างมากที่สุดหนึ่งครั้งก่อนจะหายไปตลอดกาล (เมื่อมีการ VISIT หน้าใหม่ทับ) ดังนั้นเมื่อคิดเฉลี่ยตลอดลำดับ operation ทั้งหมด งานรวมในการ push และ pop จะแปรผันเชิงเส้นกับจำนวน operation ทั้งหมด ทำให้ต้นทุนเฉลี่ยต่อ operation คือ $O(1)$

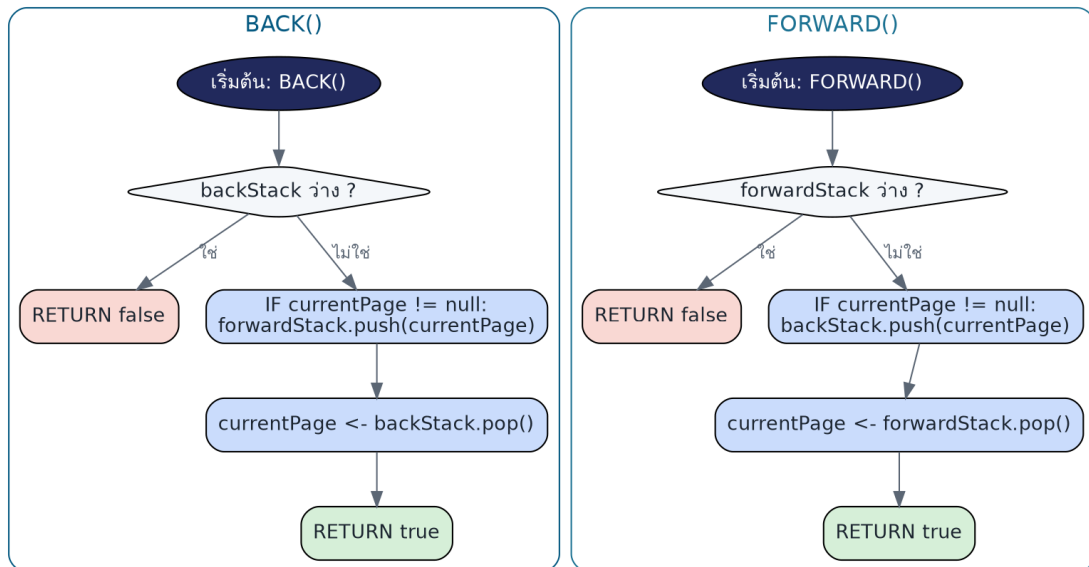
7. Flowchart และ UML Class Diagram

7.1 Flowchart: VISIT()

แสดงลำดับขั้นตอนของ VISIT ซึ่งเป็นฟังก์ชันที่ซับซ้อนที่สุด (ต้องพิจารณาทั้งการ push หน้าปัจจุบันเดิม และการล่าง forwardStack ทั้งหมด)

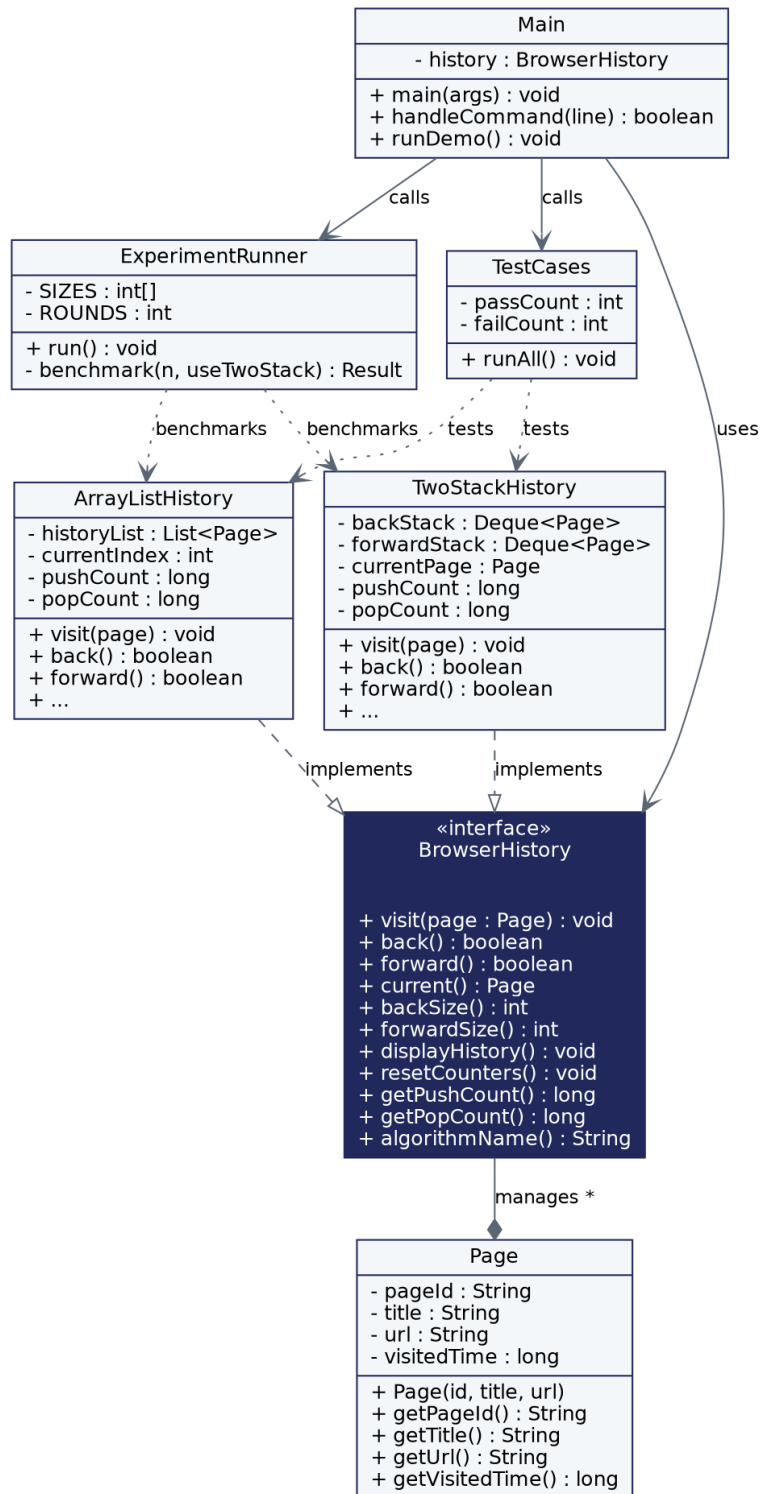


7.2 Flowchart: BACK() และ FORWARD()



7.3 UML Class Diagram

BrowserHistory เป็น interface กลางที่ TwoStackHistory (Algorithm A) และ ArrayListHistory (Algorithm B) implement ร่วมกัน ทำให้ Main, TestCases และ ExperimentRunner เรียกใช้งานทั้งสองอัลกอริทึมผ่านหน้าตาเดียวกัน (Polymorphism) และเปรียบเทียบผลลัพธ์/ประสิทธิภาพได้อย่างยุติธรรม



8. การพัฒนาโปรแกรมภาษา Java

8.1 โครงสร้างคลาส

คลาส	หน้าที่
Page	Data Model Class เก็บ Page ID, Title, URL, Visited Time
BrowserHistory (interface)	กำหนดสัญญาการทำงานร่วมของ Algorithm A และ B (visit, back, forward, current, displayHistory, ...)
TwoStackHistory	Algorithm A — ใช้ ArrayDeque<Page> สองตัวเป็น backStack และ forwardStack
ArrayListHistory	Algorithm B — ใช้ ArrayList<Page> และ currentIndex
Main	เมนูรับคำสั่ง (VISIT/BACK/FORWARD/CURRENT/DISPLAY/SWITCH/DEMO/TEST/EXPERIMENT), ตรวจสอบ Input
TestCases	รัน Test Cases บังคับ 8 กรณี เปรียบผลลัพธ์ระหว่าง Algorithm A และ B
ExperimentRunner	รันการทดลองประสิทธิภาพ $n = 1,000/10,000/50,000/100,000$ เฉลี่ย 5 รอบ บันทึกผลเป็น CSV

8.2 การตรวจสอบ Input และ Stack ว่าง

- VISIT ต้องระบุ pageId ที่ไม่ว่าง มิฉะนั้นแจ้ง [Input Validation] และไม่ทำให้โปรแกรมล่ม
- BACK/FORWARD ตรวจสอบ isEmpty() ก่อนเสมอ คืนค่า false แทนการโยน Exception เมื่อ Stack ว่าง
- คำสั่งที่พิมพ์ผิดจะถูกดักจับด้วย try-catch ในเมนูหลักและแสดงข้อความแจ้งเตือนโดยไม่ทำให้โปรแกรมหยุดทำงาน

8.3 วิธียคอมไพล์และรันโปรแกรม

```
javac *.java -d out
cd out
java Main
> DEMO          (รันชุดคำสั่งบังคับ)
> TEST          (รัน Test Cases 8 กรณี เปรียบ A กับ B)
> EXPERIMENT    (รันการทดลองประสิทธิภาพ บันทึก CSV)
> SWITCH A      (สลับไปใช้ Algorithm A)
> SWITCH B      (สลับไปใช้ Algorithm B)
```

9. Test Cases และผลการทดสอบ

รันจริงผ่านคำสั่ง TEST ในโปรแกรม ทดสอบทั้ง Algorithm A และ B ด้วย input ชุดเดียวกันทุกกรณี ผลลัพธ์ทั้งหมด PASS = 18/18 (8 กรณี × 2 อัลกอริทึม + การตรวจสอบสถานะเริ่มต้น)

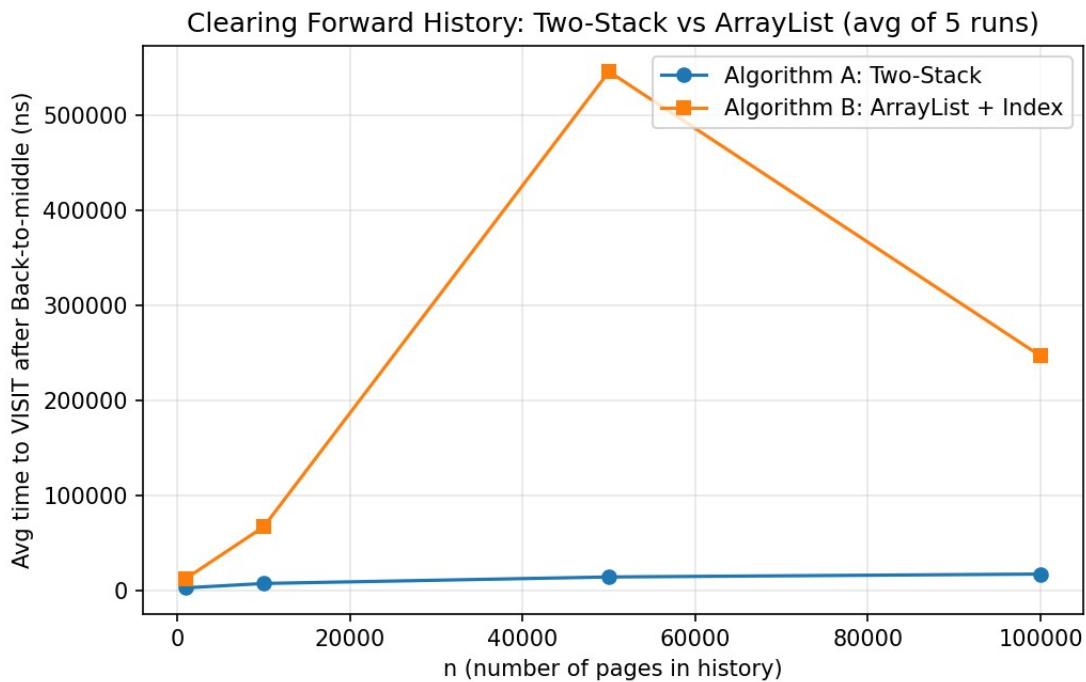
#	Test Case	ผลที่คาดหวัง	ผล A: Two-Stack	ผล B: ArrayList
1	Back เมื่อไม่มีหน้าก่อน	back()=false, current ไม่	PASS	PASS

#	Test Case	ผลที่คาดหวัง	ผล A: Two-Stack	ผล B: ArrayLi st
	หน้า	เปลี่ยน		
2	Forward เมื่อไม่มีหน้าถัดไป	forward() $\neq$ false, current ไม่เปลี่ยน	PASS	PASS
3	เปิดหน้าแรก	current=หน้านั้น, backSize=0	PASS	PASS
4	เปิดหน้าเดิมซ้ำ	backSize เพิ่มขึ้น 1	PASS	PASS
5	Back หลายครั้ง	current ย้อนกลับถูกต้อง, forwardSize เพิ่มขึ้นตาม	PASS	PASS
6	Forward หลายครั้ง	current กลับมาที่เดิม, forwardSize=0	PASS	PASS
7	Back แล้วเปิดหน้าใหม่	Forward History ถูกล้างทั้งหมด	PASS	PASS
8	ประวัติจำนวนมาก (n=20,000)	current/backSize ถูกต้องตามจำนวน Back	PASS	PASS

10. ผลการทดลองเปรียบเทียบประสิทธิภาพ

การทดลอง: สร้างประวัติขนาด n หน้า $\rightarrow$ Back ไปยังตำแหน่งกึ่งกลาง (สร้าง Forward History ขนาด n/2) $\rightarrow$ VISIT หน้าใหม่ 1 หน้า แล้ววัดเวลาเฉพาะขั้นตอน VISIT นี้ (ซึ่งรวมการล้าง Forward History) ทำซ้ำ 5 รอบต่อขนาดข้อมูล แล้วรายงานค่าเฉลี่ย

n	Algorithm	Avg Time (ns)	Push	Pop	จำนวนที่ถูก ล้าง (Forward)
1,000	A: Two-Stack	3,028.2	1	500	500
1,000	B: ArrayList	12,934.2	1	500	500
10,000	A: Two-Stack	7,594.2	1	5000	5000
10,000	B: ArrayList	67,297.4	1	5000	5000
50,000	A: Two-Stack	14,434	1	25000	25000
50,000	B: ArrayList	546,380.8	1	25000	25000
100,000	A: Two-Stack	17,505	1	50000	50000
100,000	B: ArrayList	247,073.4	1	50000	50000



10.1 อภิปรายผล เทียบกับ Big-O

จำนวน Push/Pop ที่วัดได้ (เช่น pop = 500 ที่ n=1,000 และ pop = 50,000 ที่ n=100,000) เพิ่มขึ้นเป็นเส้นตรงตามขนาดของ Forward History ที่ถูกล้าง (n/2) ซึ่งสอดคล้องกับ Worst-case $O(n)$ ของ VISIT ตามที่วิเคราะห์ไว้ในหัวข้อ 6

อย่างไรก็ตาม เมื่อเทียบเวลาจริงระหว่างสองอัลกอริทึมที่มี Order เดียวกันคือ $O(n)$ พบว่า Algorithm A (Two-Stack) เร็วกว่า Algorithm B (ArrayList) อย่างชัดเจนในทุกขนาดข้อมูล (เร็วกว่าประมาณ 4-35 เท่า) ทั้งที่ทั้งคู่มี Big-O เท่ากัน แสดงให้เห็นว่า Big-O บอกเพียงอัตราการเติบโต ไม่ได้บอกค่าคงที่ (constant factor) ของการทำงานจริง ซึ่งในที่นี้เกิดจาก `ArrayDeque.clear()` ของ Java ทำงานแบบรีเซ็ตตัวชี้ต้น-ท้ายภายในอาร์เรย์วงกลม (circular buffer) ในขณะที่ `ArrayList.subList().clear()` ต้องเรียก `System.arraycopy` และปรับขนาดโครงสร้างภายในซึ่งมี overhead สูงกว่า

สรุป: ผลการทดลองสอดคล้องกับ Big-O ที่วิเคราะห์ไว้ (ทั้งคู่เติบโตแบบ $O(n)$ ตามขนาด Forward History) แต่ค่าคงที่ของ Two-Stack ต่ำกว่าอย่างมีนัยสำคัญในการทดลองนี้

11. คำถามวิเคราะห์

1) Invariant ของ Back Stack และ Forward Stack คืออะไร

Back Stack: element ทุกตัวเรียงตามลำดับเวลาการเข้าชมจริงก่อน `currentPage` โดยตัวบนสุดคือหน้าที่เข้าชมล่าสุดก่อนหน้าปัจจุบัน — Forward Stack: element ทุกตัวคือหน้าที่เคยเป็น `currentPage` มาก่อนแล้วถูก Back ออกไป เรียงจากตัวที่ถูก Back ออกล่าสุดอยู่บนสุด ทั้งสอง Stack ไม่มี element ซ้ำกับ `currentPage` ปัจจุบัน

2) เหตุใด Forward History ต้องถูกล้างเมื่อ Visit หน้าใหม่

เพราะ Forward History แทนสายการเข้าชม (branch) เดิมที่ผู้ใช้เคยไปถึงมาก่อน เมื่อผู้ใช้เปิดหน้าใหม่จากตำแหน่งที่ Back มา ถือเป็นการสร้างสายใหม่ (new branch) สายเดิมจึงไม่ถูกต้องอีกต่อไปและต้องถูกทิ้ง เช่นเดียวกับพฤติกรรมจริงของ Browser ทัวไป

3) Two-Stack และ ArrayList แตกต่างกันอย่างไรเมื่อเปิดหน้าใหม่หลัง Back

Two-Stack ล้าง forwardStack ทั้งหมดด้วย pop วนซ้ำ (หรือ clear()) ซึ่งภายในทำงานคล้ายกัน) ส่วน ArrayList ตัด (truncate) ช่วงท้ายของ list ที่เกิน currentIndex ทั้งด้วย subList().clear() ทั้งสองวิธีมี Time Complexity เดียวกันคือ $O(k)$ โดย k คือขนาด Forward History แต่ผลการทดลองแสดงว่า ArrayList มี overhead ในการจัดการหน่วยความจำสูงกว่า Two-Stack อย่างชัดเจน

4) Amortized $O(1)$ แตกต่างจาก Worst-case $O(n)$ อย่างไร

Worst-case $O(n)$ พิจารณาการทำงานที่แย่ที่สุดของ operation เดียว ณ ขณะใดขณะหนึ่ง (เช่น VISIT ที่ต้องล้าง Forward History ขนาดใหญ่มากในครั้งเดียว) ส่วน Amortized $O(1)$ พิจารณาต้นทุนเฉลี่ยต่อ operation เมื่อคิดรวมตลอดลำดับ operation ทั้งหมด เนื่องจากแต่ละ element ถูกล้างทิ้งได้เพียงครั้งเดียวตลอดอายุการใช้งาน งานรวมทั้งหมดจึงไม่เกิน $O(\text{จำนวน operation ทั้งหมด})$ ทำให้ต้นทุนเฉลี่ยต่อครั้งคือ $O(1)$ แม้บางครั้งจะมี operation เดียวที่ใช้เวลานานกว่าปกติ

5) วิธีใดเหมาะกับ Browser History มากกว่า

จากผลการทดลอง Two-Stack เหมาะสมกว่าสำหรับกรณีนี้ เนื่องจากให้เวลาทำงานจริงที่เร็วกว่าอย่างสม่ำเสมอในทุกขนาดข้อมูลแม้จะมี Order เดียวกัน อีกทั้งโครงสร้างยังสอดคล้องโดยตรงกับพฤติกรรม Back/Forward ที่เป็น LIFO ตามธรรมชาติ อย่างไรก็ตาม หากระบบต้องการ random access เข้าถึงหน้ากลาง ๆ ของประวัติโดยไม่ผ่าน Back/Forward ที่ละชั้น (เช่นแสดงรายการประวัติทั้งหมดพร้อม index) ArrayList จะสะดวกกว่า

12. บันทึกการใช้ Generative AI

ส่วนนี้เป็นแบบฟอร์มเริ่มต้นสำหรับบันทึกการใช้ Generative AI ตามข้อกำหนดของงาน สมาชิกกลุ่มทุกคนต้องช่วยกันกรอกข้อมูลจริงให้ครบถ้วนก่อนส่งงาน โดยเฉพาะช่อง 'วิธีตรวจสอบ', 'สิ่งที่แก้ไข' และ 'Reflection' ซึ่งต้องเป็นสิ่งที่นักศึกษาทำเองจริง เพราะผู้สอนอาจสุ่มถามสมาชิกคนใดก็ได้เกี่ยวกับส่วนนี้

งานที่ใช้ AI ช่วย	Prompt ที่ใช้ (สรุป)	ผลลัพธ์จาก AI	วิธีตรวจสอบ (ต้องกรอกเอง)	สิ่งที่แก้ไข (ต้องกรอกเอง)
วิเคราะห์ปัญหาและออกแบบ Pseudocode 2 วิธี	ขอวิเคราะห์ Input/Output/ Constraints และออกแบบอัลกอริทึม Two-Stack กับ ArrayList+Index สำหรับ	โครงสร้างการวิเคราะห์ปัญหาและ Pseudocode ทั้งสองวิธี	[]	[]

งานที่ใช้ AI ช่วย	Prompt ที่ใช้ (สรุป)	ผลลัพธ์จาก AI	วิธีตรวจสอบ (ต้องกรอกเอง)	สิ่งที่แก้ไข (ต้องกรอกเอง)
	Browser Navigation			
เขียนโปรแกรม Java	ขอให้พัฒนาโปรแกรม Java ตามโครงสร้างคลาสที่กำหนด (Page, BrowserHistory, TwoStackHistory, ArrayListHistory, Main, TestCases, ExperimentRunner)	ซอร์สโค้ด Java ที่คอมไพล์และรันผ่าน	[]	[]
ออกแบบ Test Cases	ขอให้สร้าง Test Cases ครอบคลุม 8 กรณีตามที่โจทย์กำหนด และรันเทียบผลทั้งสองอัลกอริทึม	Test Cases 8 กรณี ผลลัพธ์ PASS ทั้งหมด	[]	[]
Correctness Argument	ขอให้อธิบายความถูกต้องของ Algorithm A ด้วย Precondition/Postcondition และ Loop Invariant	ร่างคำอธิบายความถูกต้อง 5 ประเด็นตามที่โจทย์กำหนด	[]	[]
วิเคราะห์ Big-O	ขอให้วิเคราะห์ Time/Space Complexity ของทุกฟังก์ชัน ไม่ใช่แค่ push/pop	ตาราง Best/Average/Worst Case และ Auxiliary Space	[]	[]

- การวิเคราะห์ Big-O ของนักศึกษาเอง: [ให้สมาชิกอธิบายด้วยคำพูดตนเองเพิ่มเติมว่าทำไมผลการทดลองจริงจึงสอดคล้อง/ไม่สอดคล้องกับ Big-O ที่วิเคราะห์ไว้]
- Reflection เกี่ยวกับข้อดีและข้อจำกัดของ AI: [ให้สมาชิกเขียนความเห็นของตนเองเกี่ยวกับสิ่งที่ AI ช่วยได้ดีและสิ่งที่ต้องตรวจสอบเพิ่มเติมด้วยตนเอง]

13. สรุป

กลุ่มที่ 2 ได้พัฒนาอัลกอริทึม 2 วิธีสำหรับระบบ Browser Navigation คือ Two-Stack Method และ ArrayList with Current-Index Method ทั้งสองวิธีแก้ปัญหเดียวกันและให้ผลลัพธ์ถูกต้องตรงกันทุก Test Case โดยมี Time Complexity ระดับเดียวกัน ($O(1)$ amortized สำหรับ VISIT, $O(1)$ worst-case สำหรับ BACK/FORWARD) แต่ผลการทดลองจริงแสดงให้เห็นว่า Two-Stack มีค่าคงที่

(constant factor) ที่ต่ำกว่าอย่างชัดเจน ทำให้เหมาะสมกว่าสำหรับการใช้งานจริงในกรณีนี้